

Smart Attendance System — Technical Specification v8 (Build-Ready)

Purpose of this document: Complete build specification for a multi-tenant, automated classroom attendance system. Version 8 adds a "permitted exit" mechanism: when a student genuinely needs to step out mid-class (emergency, family matter, etc.) and informs the teacher, the teacher can mark it with one tap so the system never flags that window as a suspected proxy exit. Hand this document to an AI coding assistant (Claude Code, Cursor, etc.) and build phase by phase — do not implement everything in one pass.

1. Product Summary

A multi-tenant SaaS platform that automates classroom attendance using face recognition, eliminating proxy attendance without requiring manual effort from teachers. One codebase serves unlimited institutions, each fully data-isolated. Performance stays linear regardless of institution size.

Non-negotiable qualities:

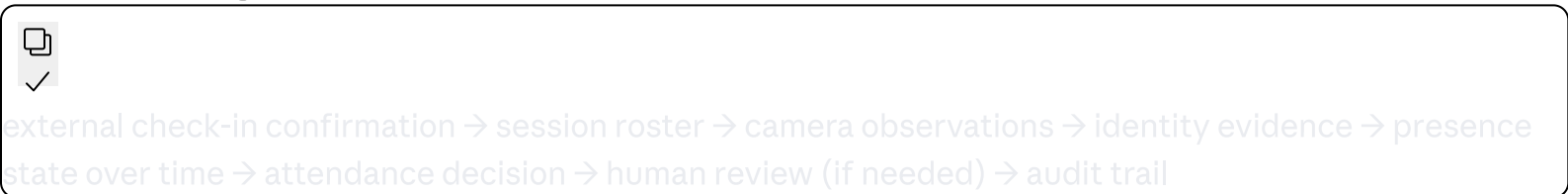
Zero *routine* teacher interaction during class (the teacher may still need to intervene on genuine system issues — see Section 8)

No single point of failure — a camera outage or one failed face match never crashes a session or silently loses data

Linear performance regardless of institution size (matching is always scoped to a single class roster, never the whole institution)

Every automated decision is reversible and auditable

Mental model (this is the key shift from v1): the system does not go straight from "face recognition" to "attendance." It goes:



Treating raw detections as *evidence*, not as the final answer, is what makes the rest of this document work. As of v6, the system also does not assume every enrolled student needs to be watched for — it only continuously monitors students the college's existing entry-scan device (referred to below as the "external check-in system") has already confirmed as present for that session. This keeps the product from duplicating hardware/verification the institution has already paid for, and shrinks the matching workload to a smaller, more accurate candidate pool per session.

2. Tech Stack

Layer	Choice	Why
Frontend	Next.js (App Router) + TypeScript	Type safety, SSR for dashboard
Hosting (frontend)	Vercel	Matches Next.js natively
Database	Supabase (Postgres) + <code>pgvector</code>	RLS gives multi-tenancy for free; native vector search for embeddings
Face processing	Python (FastAPI) + <code>face_recognition</code> (dlib) for v1	Mature, well-documented — lower hallucination risk for AI-generated code. Wrapped behind an internal interface so it can be swapped later without touching the rest of the app (see Section 6)
Face processing hosting	Railway or Render	Supports long-running background workers
Queue	Postgres-based queue table (decided — do not leave as an open choice)	Avoids adding Redis as a second infra dependency while the team is small; revisit only if queue throughput becomes a measured bottleneck
Auth	Supabase Auth	Built-in role-based access (student/teacher/admin)
Payments	Razorpay	Best fit for Indian institutions
File/photo storage	Supabase Storage	Same RLS model as the database

Rule for the AI coding assistant: no new library or service outside this table without explicitly flagging the deviation and the reason. Every added dependency is a new failure and hallucination surface.

3. Multi-Tenancy Model (build before any feature)

Every table carries `institution_id`. RLS policies enforce isolation at the **database level**, not just in application code.



```
sql
create policy "institution_isolation" on students
for all
using (institution_id = (select institution_id from users where id = auth.uid()));
```

Apply this pattern to every table without exception.

4. Database Schema (v2 — corrected)



sql

```
create extension if not exists vector;
```

```
create table institutions (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  plan_type text not null default 'trial',  
  subscription_status text not null default 'active',  
  created_at timestamptz default now()  
);
```

```
create table users (  
  id uuid primary key references auth.users(id),  
  institution_id uuid references institutions(id) not null,  
  role text not null check (role in ('admin','teacher','student')),  
  full_name text not null  
);
```

```
create table rooms (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  name text not null,  
  block text,  
  floor text  
);
```

```
-- Cameras separated from rooms: a room may get a second camera later  
-- without a schema migration.  
-- RTSP URLs typically embed credentials (rtsp://user:pass@ip:port/stream).  
-- Never store that raw string in a normal table column. Store only a  
-- reference to a secret; the actual credential lives in the hosting  
-- platform's secret manager (Railway/Render env vars) or a dedicated  
-- encrypted secrets table with column-level encryption. The application  
-- resolves credential_ref -> actual RTSP URL only inside the worker  
-- process, never in a value that could appear in a dashboard API response  
-- or a log line.
```

```
create table cameras (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  room_id uuid references rooms(id) not null,  
  host text not null,  
  stream_path text not null,  
  credential_ref text not null, -- key name in secret manager, not the credential itself  
  label text default 'primary',  
  is_active boolean default true
```



```
);
```

-- status added: a student isn't just "exists" — they can be inactive
-- (semester break), graduated, or transferred, all of which must stop
-- matching without deleting historical attendance records.

```
create table students (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  user_id uuid references users(id),  
  status text not null default 'active'  
  check (status in ('active','inactive','graduated','transferred')),  
  enrollment_photo_count int default 0  
);
```

-- One row PER ENROLLMENT PHOTO, not one embedding per student.
-- A single photo produces a fragile embedding (one angle, one lighting
-- condition). Storing each of the 4-5 enrollment photos as its own row
-- lets matching compare against the best available representation
-- (or an average) instead of one brittle vector. is_primary marks the
-- one used for quick previews in the dashboard.

```
create table student_biometrics (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  student_id uuid references students(id) not null,  
  face_embedding vector(128),  
  embedding_model text not null default 'dlib_resnet_v1',  
  embedding_version int not null default 1,  
  is_primary boolean default false,  
  quality_score float,  
  created_at timestamptz default now()  
);
```

```
create index on student_biometrics using ivfflat (face_embedding vector_cosine_ops);
```

```
create table classes (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  room_id uuid references rooms(id) not null,  
  subject text not null,  
  teacher_id uuid references users(id),  
  recurrence text, -- e.g. 'MON,WED,FRI 10:00-11:00'  
  is_active boolean default true  
);
```

-- Many-to-many, replaces the v1 array column. Supports section changes,

-- drops, late joins, and clean historical reporting.

```
create table class_enrollments (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  class_id uuid references classes(id) not null,  
  student_id uuid references students(id) not null,  
  status text not null default 'active' check (status in ('active','dropped')),  
  enrolled_at timestamptz default now(),  
  unique(class_id, student_id)  
);
```

-- One row per actual occurrence of a class, not per timetable slot.

-- This is what everything else (observations, results, camera health)

-- attaches to.

-- Raw check-in events pulled from the college's existing entry-scan

-- device (e.g. Kent). This is a separate source of truth from anything

-- our own cameras observe — it tells us WHO to watch for in a session,

-- not how long they stayed. Populated by a sync job (Section 5, Phase A.5)

-- via API polling or scheduled CSV import, whichever the vendor/college

-- supports.

```
create table external_checkin_events (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  source text not null default 'kent', -- vendor identifier, kept generic in case the device changes  
  external_student_ref text not null, -- the ID the external system uses, mapped to our student_id below  
  student_id uuid references students(id), -- resolved during sync; null if the ID couldn't be matched yet  
  checked_in_at timestamptz not null,  
  session_id uuid references class_sessions(id), -- resolved by matching checked_in_at against session time  
  raw_payload jsonb, -- store whatever the vendor sends, for debugging mapping issues  
  synced_at timestamptz default now(),  
  unique(source, external_student_ref, checked_in_at)  
);
```

```
create table class_sessions (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  class_id uuid references classes(id) not null,  
  camera_id uuid references cameras(id),  
  scheduled_start timestamptz not null,  
  scheduled_end timestamptz not null,  
  -- These are NOT set by a teacher action — they're derived automatically  
  -- during finalization via quorum detection (Section 5, Phase E, step 0).  
  -- actual_start/actual_end represent when the class was genuinely in  
  -- session, which can differ from the timetable if the teacher started
```

```

-- late or ended early. Null until finalization runs; falls back to
-- scheduled_start/scheduled_end if quorum was never reached (e.g. a
-- cancelled class, or a camera outage covering the whole session).
actual_start timestampz,
actual_end timestampz,
status text not null default 'scheduled'
check (status in ('scheduled','in_progress','completed','cancelled')),
camera_status text default 'unknown'
check (camera_status in ('unknown','healthy','degraded','offline')),
processing_status text default 'pending'
check (processing_status in ('pending','processing','finalized','failed')),
-- Was the session's monitoring roster built from external check-in
-- data, or (fallback) from the full class_enrollments list? Recorded
-- explicitly so a finalization bug or a sync outage is visible in the
-- data, not silently assumed.
roster_source text not null default 'external_checkin'
check (roster_source in ('external_checkin','full_enrollment_fallback')),
-- Set explicitly the moment finalization starts, and checked again
-- before writing results. If a retry or duplicate job finds this
-- already set, it exits without recomputing — this is what makes
-- finalization idempotent rather than "probably fine because of the
-- unique constraint downstream."
finalized_at timestampz
);

```

```

-- Every capture attempt — regardless of whether a usable frame came back.
-- This is distinct from attendance_observations: a capture_event records
-- "did the camera give us a frame", an observation records "what did we
-- find in that frame". Separating them means a camera outage shows up
-- immediately in its own record, not inferred later from a gap in
-- observations.

```

```

create table capture_events (
id uuid primary key default gen_random_uuid(),
institution_id uuid references institutions(id) not null,
session_id uuid references class_sessions(id) not null,
camera_id uuid references cameras(id) not null,
attempted_at timestampz not null,
succeeded boolean not null,
error text,
frame_stored boolean default false,
unique(session_id, attempted_at)
);

```

```

-- Raw evidence. Never overwritten — the final attendance status is a

```

-- computed view derived from these rows, not the source of truth.

```
create table attendance_observations (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  session_id uuid references class_sessions(id) not null,  
  student_id uuid references students(id),  
  captured_at timestamptz not null,  
  similarity_score float,  
  quality_score float,  
  -- unknown_face added: a face was detected and clear enough to embed,  
  -- but didn't match anyone in the class roster. This is a different  
  -- situation from "no_face" (nothing to analyze) or "low_confidence"  
  -- (a candidate match that isn't trusted) — an unknown_face could be a  
  -- visiting student, someone from a neighbouring section, or a genuine  
  -- proxy attempt, and it's worth surfacing separately in review rather  
  -- than silently discarding it.  
  match_status text not null  
  check (match_status in ('matched','low_confidence','no_face','poor_quality','unknown_face','camera_failure')),  
  evidence_photo_url text,  
  model_version text,  
  unique(session_id, student_id, captured_at)  
);
```

```
create table final_attendance (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  session_id uuid references class_sessions(id) not null,  
  student_id uuid references students(id) not null,  
  presence_score float,  
  status text not null  
  check (status in ('present','absent','left_early','uncertain','camera_issue')),  
  -- true if a session_exceptions window influenced this result (excluded  
  -- from scoring/gap-check). Kept as a simple flag rather than baking  
  -- exception details into final_attendance itself — the full record  
  -- lives in session_exceptions and is reachable via session_id + student_id.  
  exception_applied boolean default false,  
  finalized_at timestamptz default now(),  
  unique(session_id, student_id)  
);
```

-- A teacher-recorded exception for a student's genuine mid-class exit
-- (emergency, family matter, sent on an errand, etc.). This is the
-- *only* routine-adjacent teacher action the product asks for — a single
-- tap, not a form — and it exists precisely so the automated gap-check

-- (Section 5, Phase E, step 8) never has to guess about a legitimate
-- absence. return_at is optional: the teacher may not know exactly when
-- the student will be back, and doesn't need to track it — the window
-- this exception covers is simply excluded from scoring, whether the
-- student returns or not.

```
create table session_exceptions (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  session_id uuid references class_sessions(id) not null,  
  student_id uuid references students(id) not null,  
  marked_by uuid references users(id) not null, -- must be a teacher/admin, enforced by RLS  
  reason text, -- optional free text, e.g. "family emergency", "sent to office"  
  exit_at timestampz not null default now(),  
  return_at timestampz, -- nullable; teacher can fill in later, or never, if student didn't return  
  created_at timestampz default now()  
);
```

```
create table disputes (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  final_attendance_id uuid references final_attendance(id) not null,  
  student_id uuid references students(id) not null,  
  reason text,  
  status text default 'pending' check (status in ('pending','approved','rejected')),  
  evidence_photo_url text,  
  created_at timestampz default now(),  
  resolved_at timestampz  
);
```

-- Every status change a human makes is recorded. Cheap to add now,
-- valuable for institutional trust and dispute resolution later.

```
create table audit_logs (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  actor_user_id uuid references users(id),  
  action text not null,  
  entity_type text not null,  
  entity_id uuid not null,  
  old_value jsonb,  
  new_value jsonb,  
  created_at timestampz default now()  
);
```

```
create table camera_health (  
  id uuid primary key default gen_random_uuid(),  
  institution_id uuid references institutions(id) not null,  
  camera_id text not null,  
  status text not null,  
  last_checked_at timestampz not null,  
  created_at timestampz default now()  
);
```

```

camera_id uuid primary key references cameras(id),
last_frame_at timestamptz,
consecutive_failures int default 0,
status text default 'unknown' check (status in ('unknown','healthy','degraded','offline')),
last_error text
);

```

-- Actual storage for the thresholds mentioned in Phase E, not just a
 -- promise in prose that they're "versioned." One active row per
 -- institution (or a global default row when institution_id is null).
 -- Finalization reads this table, never a hardcoded constant.

```

create table attendance_config (
id uuid primary key default gen_random_uuid(),
institution_id uuid references institutions(id), -- null = platform default
present_threshold float not null default 0.70,
left_early_threshold float not null default 0.20,
min_valid_observations int not null default 3, -- see Section 5, Phase E
max_gap_minutes int not null default 10, -- see Section 5, Phase E — mid-class exit flagging
-- Quorum detection settings (see Section 5, Phase E, step 0): a sample
-- round counts as "class was genuinely in session" if at least this
-- fraction of the roster (or the absolute minimum count, whichever is
-- higher) was matched in that round.
quorum_fraction float not null default 0.30,
min_quorum_count int not null default 3,
-- How far outside the timetabled window capture continues, so a late
-- start or extended session still gets observed (see Section 5, Phase D).
capture_buffer_minutes int not null default 15,
version int not null default 1,
created_at timestamptz default now(),
is_active boolean default true
);

```

-- Aggregate, low-cardinality counters for operational monitoring —
 -- how many frames processed, average recognition latency, match rate
 -- per session. Not per-observation detail (that's attendance_observations);
 -- this is what a monitoring dashboard queries so it isn't scanning raw
 -- evidence tables.

```

create table processing_metrics (
id uuid primary key default gen_random_uuid(),
institution_id uuid references institutions(id) not null,
session_id uuid references class_sessions(id),
frames_attempted int default 0,
frames_succeeded int default 0,
avg_processing_time_ms float,

```

```
recorded_at timestamptz default now()
```

```
;
```

5. End-to-End Workflow

Phase A.5 — External check-in sync (new in v6, runs continuously)

Before this can work at all, the college needs to actually give access to the check-in data — this is a real-world conversation with the college IT admin / device vendor, not just a technical step. Ask for, in order of preference: (a) an API or "push" feed if the device supports one, (b) a scheduled export (CSV/Excel) from the admin console if no API exists. Get clarity on exactly what fields are available — at minimum a student identifier and a timestamp. If the device also exports a photo per check-in, that's a bonus that could later remove the need for our own enrollment photos entirely (see Section 11 — kept deferred for now since your college's device currently returns timestamp-only data)

A sync job (polling the API on a short interval, or importing the latest export on a schedule) writes new events into `external_checkin_events`

Each event's `external_student_ref` is resolved against our `students` table (a one-time mapping step during onboarding — see Section 10, Phase 1.5) and each event's `checked_in_at` is matched against `class_sessions` time windows to set `session_id`

If a session's start time arrives and its expected check-in data hasn't synced yet (API down, export late), the session falls back to `roster_source = 'full_enrollment_fallback'` — monitoring proceeds against the full `class_enrollments` list rather than blocking the session entirely. This keeps a vendor outage from ever preventing attendance tracking

Phase B — Setup (one-time per institution)

Institution signs up → `institutions` row created

Admin bulk-uploads students (CSV + photos) → background job runs quality checks, generates embeddings, writes to `student_biometrics`

Admin enters timetable → `classes` populated; students mapped via `class_enrollments`

Admin registers cameras against rooms

(New) Admin maps the external check-in system's student identifiers to our `students.id` — a one-time CSV mapping upload, or an ID-matching rule (e.g. roll number) if the identifiers already align

Phase C — Session creation

A scheduler generates a `class_sessions` row shortly before each `classes` recurrence is due to start

When the session's start time arrives, status moves to `in_progress`. The session's monitoring roster is derived: `class_enrollments` for that class, intersected with any `external_checkin_events` already synced for that session window (Phase A.5). If no check-in data is available yet, the fallback in Phase A.5 step 4 applies

Phase D — Live capture (fully automated)

Scheduler pushes a "capture frame" job to the queue every 5–8 minutes (randomized, computed by the scheduler up front for the session — not a `sleep()` inside a worker). **Capture starts a buffer period**

before `scheduled_start` and continues a buffer period after `scheduled_end` (default 15 minutes each side, configurable) — this is what makes quorum detection (Phase E, step 0) actually able to catch a late start or an early-then-extended session; without this buffer, a class starting 20 minutes late would have its first 20 minutes of real activity never captured at all

A worker pulls the job, grabs one frame from the camera's RTSP stream

If the frame can't be captured, `camera_health` is updated (`consecutive_failures += 1`) and the job is skipped — this never blocks the session

If capture succeeds, a "process frame" job is queued

A recognition worker runs face detection + quality check, then matches only against `student_biometrics` for students **in that session's monitoring roster** (Phase C) — this is now a narrower set than "everyone enrolled in the class," which is both faster and reduces false-positive matches against students who never checked in at all

Every attempt — match, low-confidence, no-face, poor-quality — is written to `attendance_observations`.

Failures are evidence too, not silence

If a student informs the teacher they need to step out for a genuine reason, the teacher taps "Mark permitted exit" in the live session view — a `session_exceptions` row is created with `exit_at` (defaults to now) and an optional reason and `return_at`. This is the one exception to "zero routine teacher interaction" (Section 8) — it's optional, rare, and takes one tap, not a workflow

Phase E — Finalization (automatic at session end, explicitly idempotent)

Session status moves to `completed`; a finalize job runs

Idempotency guard first: the job checks `class_sessions.finalized_at`. If already set, it exits immediately — no recomputation, no duplicate writes. This is checked and set atomically (a single `update ... where finalized_at is null returning id` — if no row comes back, another worker already claimed it) so two concurrent finalize attempts (a retry, a redelivered queue message) can never both proceed

Step 0 — determine the session's actual boundaries (quorum detection): group this session's `matched` observations by capture round (they're already roughly aligned since capture rounds are scheduler-driven). For each round, count distinct students matched. Reading `quorum_fraction` and `min_quorum_count` from `attendance_config`, find the earliest round where the matched count meets `max(quorum_fraction × roster_size, min_quorum_count)` — that round's timestamp becomes `actual_start`. Find the latest such round — that becomes `actual_end`. If no round ever reaches quorum (e.g. the camera was down the whole session, or the roster itself was tiny and noisy), fall back to `scheduled_start`/`scheduled_end` and flag the session `camera_issue` for review rather than guessing further

All subsequent steps in this phase use `actual_start`/`actual_end`, not the timetabled times — this is the fix for late starts and early ends: time before `actual_start` or after `actual_end` is excluded from every student's presence calculation entirely, it's simply not counted as class time

If `camera_health` shows the camera was offline/degraded for a meaningful portion of the (now-corrected) session window, affected students are marked `camera_issue` rather than `absent` or `uncertain` — this is a distinct, surfaced state, not folded into a generic catch-all

Minimum coverage check: read `min_valid_observations` from `attendance_config`. If a student has fewer valid observations than this floor within the actual session window, their result is `uncertain` regardless of their raw match ratio

Otherwise, `presence_score` is computed from valid observations within `[actual_start, actual_end]` only (frames during a camera outage don't count against a student either)

Apply permitted exits before the gap check: for each student, fetch any `session_exceptions` rows for this session. Any time window covered by an exception (`exit_at` to `return_at`, or `exit_at` to `actual_end` if `return_at` is null) is excluded from both the `presence_score` denominator and the gap-check in the next step — it's treated the same way a camera outage is treated, as time that simply doesn't count against the student. Set `final_attendance.exception_applied = true` for this student

Gap check (mid-class exit detection, quality-aware): for each student, walk their observations in time order and find the largest gap between consecutive `matched` results — and separately, the gap between their last match and `actual_end`. Gaps that fall entirely within an applied exception window (step 8) are skipped — they were already accounted for and must never trigger `left_early`. For any remaining gap, look at what happened *during* it:

If the observations inside the gap are mostly `no_face` / `poor_quality` on **clean, well-lit captures** (the camera clearly had a view of the room and simply found nothing usable there) → this is real evidence of absence. If the gap exceeds `max_gap_minutes` (default 10, from `attendance_config`), mark `left_early`. If the observations inside the gap show partial/occluded faces, or `capture_events` / `camera_health` indicate degraded conditions (poor lighting, camera angle issues) — this looks like occlusion (cap, hoodie, beard growth, seasonal appearance change), not necessarily absence. Do **not** auto-mark `left_early` in this case. Mark the student `uncertain` instead and surface it to the teacher with the evidence photos from that window, so a human makes the call in one click rather than the system guessing

This distinction is why `attendance_observations.match_status` includes `poor_quality` as its own value separate from `no_face` — it's exactly the signal this check relies on

Thresholds are read from `attendance_config` (`present_threshold`, `left_early_threshold`, `max_gap_minutes`) — never hardcoded — so they can be recalibrated per institution after the pilot without a code change

Result written to `final_attendance`; `class_sessions.finalized_at`, `actual_start`, `actual_end` set in the same transaction

Phase F — Human review (only the ambiguous minority)

`uncertain` and `camera_issue` sessions surface in the teacher dashboard with evidence photos attached. Teacher resolves in one click; every change is written to `audit_logs`

Students can raise a dispute within a 24-48 hour window with evidence shown to admin

If a teacher forgets to mark a permitted exit during class and only remembers after

`class_sessions.finalized_at` is set, the exception is still recorded in `session_exceptions` (so there's a real record of why), but it does not silently re-trigger finalization. Instead it's routed through the same dispute-resolution path as step 3 — the teacher's marked exception becomes the evidence an admin (or the teacher themselves, if permitted) uses to correct the `final_attendance` row, with the correction logged in `audit_logs`. This keeps "already-finalized" results from being quietly rewritten outside of an auditable process

Phase G — Output

CSV export always available (universal fallback, no exceptions)

Institution-specific ERP integration built only after a paying customer requests it

6. Recognition Provider Abstraction (lightweight, not premature)

Wrap the face recognition calls behind one internal interface so the dlib implementation can be replaced later without touching attendance logic, database schema, or the dashboard:



```
python
```

```
class FaceRecognitionProvider:
    def detect(self, frame) -> list[FaceBox]: ...
    def quality(self, face_box) -> float: ...
    def embed(self, face_box) -> list[float]: ...
    def match(self, embedding, candidate_embeddings) -> MatchResult: ...
```

This is a thin interface, not a plugin system — do not build configuration UI or multi-provider support in v1. It exists purely so a future model swap is a new class, not a rewrite.

7. Robustness Guardrails (non-negotiable)

No unhandled exception in capture/processing may stop the scheduler. Every job wrapped in try/catch; failures logged and skipped, never fatal to the session.

No hallucinated APIs. If unsure whether a Supabase/Next.js/FastAPI method exists, verify against documentation rather than guessing.

All matching is scoped to the session's class roster via `class_enrollments`, never the full institution table. Hard constraint, not a suggestion.

No synchronous ML inference inside API request/response cycles. All inference happens in queue workers.

Raw `attendance_observations` **are never overwritten or deleted.** Final status is a computed view.

Idempotency: the `unique(session_id, student_id, captured_at)` constraint prevents duplicate observations if a job retries.

Per-institution concurrency caps on queue workers so one large institution's load can't degrade another's session.

All timestamps stored in UTC, converted to local time only at the display layer.

Secrets (RTSP URLs, API keys) live in platform secret managers, never hardcoded or committed.

Finalization logic (Phase E) gets dedicated integration tests before anything is built on top of it — it's the highest-complexity, highest-cost-of-bug piece in the system, and must be tested for the idempotency guard specifically (simulate a duplicate finalize call and assert no duplicate rows/side effects).

Camera health is tracked continuously (`camera_health` table), not inferred after the fact from missing observations.

RTSP credentials never appear in the database, dashboard responses, or logs in plaintext — resolved from the secret manager only inside the worker process at connection time.

A student's `status` **is checked before matching** — an `inactive`/`graduated`/`transferred` student is excluded from the class-scoped candidate list even if their `class_enrollments` row wasn't cleaned up yet, so stale enrollment data can't cause a false match.

A sync outage on the external check-in system never blocks a session. If check-in data hasn't arrived by session start, the session proceeds on `roster_source = 'full_enrollment_fallback'` rather than waiting or skipping monitoring entirely — the product must keep working even if the third-party device or its export is temporarily unavailable.

`external_checkin_events.raw_payload` **is stored as-is (jsonb)** even though only a few fields are extracted into typed columns — this means if the ID-mapping logic has a bug, it can be fixed and events reprocessed from the raw payload rather than needing to re-request historical data from the college.

Presence and gap calculations always use `actual_start`/`actual_end`, **never** `scheduled_start`/`scheduled_end`, once finalization has run quorum detection (Section 5, Phase E, step 3).

Any code path that reads a session's time boundaries for scoring purposes must use the actual, derived values — the scheduled times exist for timetabling and capture-window buffering only.

`session_exceptions` **can only be created by a teacher or admin role** (enforced by RLS on `marked_by`), and every creation is implicitly auditable through the table itself plus `audit_logs` when it affects an already-finalized result (Section 5, Phase F, step 4). This is the one place a human can pre-empt an automated flag — it must never be reachable by a student marking their own exception.

Operational basics (add now — cheap, and painful to retrofit)

API and upload rate limits: bulk student/photo upload endpoints and any public-facing API routes get rate limits from day one. A single misconfigured bulk-upload script or a scraping attempt should not be able to degrade the platform for every institution.

Backup/recovery: Supabase's automatic backups must be confirmed enabled and their retention window documented before any real student data is loaded — this is a checkbox to verify, not new infrastructure to build.

Processing metrics: the `processing_metrics` table (Section 4) is written to from the queue workers so there's a basic operational view (frames processed, success rate, latency) without needing to query raw evidence tables during an incident.

8. On "zero teacher interaction"

The product promise is **zero routine teacher interaction** — the teacher never takes attendance. It is not zero interaction under all conditions: camera failures, room changes, and ambiguous cases still need a human decision point. **Marking a permitted exit for a student who genuinely needs to step out (Section 5, Phase D, step 7)** is the one additional exception — a single, optional tap, used rarely, not a routine action for every class. The dashboard should make all of these rare exceptions fast to resolve, not pretend they can't happen.

9. Privacy & Biometric Data Lifecycle

This institution is handling sensitive biometric data — treat this section as part of the architecture, not an afterthought.

Enrollment: only admins can upload/register student photos; students should be informed and, where required by institutional policy, give consent before enrollment.

Storage: `student_biometrics` holds only the embedding vector, not raw photos, once enrollment is complete — the enrollment photos themselves can be deleted after the embedding is generated, reducing what's stored long-term.

Evidence photos: kept only long enough to support the dispute window (Section 5, Phase F) — a short, configurable retention period, then deleted.

Deletion: when a student leaves the institution, their `student_biometrics` row is deleted; `final_attendance` records may be retained per the institution's own record-keeping requirements, but biometric data is not.

Access: students see only their own attendance; teachers see only their assigned classes; admins see their institution; the platform operator does not need standing access to raw biometric data for normal operation.

Do not hardcode specific retention durations — make them per-institution configuration, since legal requirements vary.

10. Build Order (phase by phase — do not implement everything at once)

Phase 0 — Foundation: multi-tenancy, RLS, auth, schema, audit logging plumbing.

Phase 1 — Enrollment: student CRUD, photo upload, quality check, embedding generation into `student_biometrics`.

Phase 1.5 — External check-in sync: build the `external_checkin_events` sync job (API polling or scheduled import, whichever the college's device supports), the one-time student-ID mapping admin flow, and the session roster derivation (intersecting `class_enrollments` with synced check-ins). Include the `full_enrollment_fallback` path from day one, not as an afterthought — a sync outage must never block a class session.

Phase 2 — Camera: camera registration, RTSP connectivity check, `camera_health` tracking, frame capture.

Phase 3 — Recognition: detection → quality → embedding → class-scoped match → `attendance_observations`, behind the provider interface from Section 6.

Phase 4 — Sessions: timetable → `class_sessions` generation → linking capture jobs to a session.

Phase 5 — Finalization: presence scoring, camera-issue handling, versioned thresholds, `final_attendance`. Write tests here before moving on.

Phase 6 — Human review: teacher dashboard for `uncertain`/`camera_issue` cases, one-click resolution, audit log writes.

Phase 7 — Disputes: student-facing dispute flow, admin resolution.

Phase 8 — Reporting: dashboard views, CSV export.

Phase 9 — Billing: trial, Razorpay subscription, plan limits.

For a team of 3, Phases 0-1 (person A), Phase 2-3 (person B), and Phase 6/dashboard shell (person C) can run in parallel once Phase 0's schema is agreed and merged. Phase 5 (finalization) should be built and tested together — it's the highest-risk logic in the system.

11. Explicitly Deferred to Phase 2 of the Product (not v1 — do not build now)

These are legitimate improvements but add ML/infra complexity disproportionate to a first pilot. Revisit only after the core loop has run reliably in a real classroom for 2-3 weeks:

Liveness/anti-spoofing detection (a separate ML problem; the existing multi-sample-per-session design already makes simple spoofing — e.g. holding up a photo — impractical to sustain across 6-8 checks)

Cross-frame face tracking (only needed for continuous video; this system uses independent periodic snapshots by design). Mid-class exit detection does **not** require this — it's achieved via the gap check in Phase E (Section 5, step 6) using the existing periodic samples. Full continuous person-tracking (frame-by-frame entry/exit detection) stays deferred until there's a concrete reason the gap-check approach isn't precise enough in real pilot data.

Formal threshold calibration pipeline (start with pilot data and manual tuning; formalize once there's enough real data to calibrate against)

Multi-camera-per-room support (schema already allows adding this later without migration pain — see `cameras` table)

Live video dashboard, ERP integrations beyond CSV, analytics, gamification, notifications, mobile apps

Using external check-in photos as the enrollment source (raised in conversation) — worth revisiting only if the college's device turns out to export a photo per check-in event. If/when that's confirmed, it could reduce or replace the manual student photo-upload step in Phase 1, but do not build this speculatively before confirming the device actually provides it

12. Summary of What Changed Across Versions

Area	v1	v2	v3	
Student-class relationship	<code>class_ids</code> <code>uuid[]</code> on students	<code>class_enrollments</code> join table	—	—
Class occurrences	Implied by timetable only	Explicit <code>class_sessions</code> per occurrence	—	—
Camera model	<code>camera_rtsp_url</code> on <code>rooms</code>	Separate <code>cameras</code> table + <code>camera_health</code>	Credentials moved out of the row entirely	—
Observation detail	<code>matched</code> <code>boolean</code> , <code>confidence_score</code>	Full <code>attendance_observations</code>	Added <code>unknown_face</code> ; capture split from recognition	—
Camera capture vs. recognition	Conflated	Conflated	Split: <code>capture_events</code> vs. <code>attendance_observations</code>	—
Ambiguous/failure states	Single <code>unverified</code> catch-all	<code>uncertain</code> and <code>camera_issue</code>	—	—
Biometric storage	One embedding per student	Same, versioned	One row per enrollment photo	—
Thresholds	Hardcoded	"Versioned" (unspecified storage)	Actual <code>attendance_config</code> table	Ad <code>ma</code>
Finalization safety	Basic unique constraint	Same	Explicit atomic idempotency guard + min-observation floor	—
Mid-class exit detection	Not modeled	Not modeled	Only implicit via overall <code>presence_score</code> ratio	Ex ch
Student lifecycle	Not modeled	Not modeled	<code>status</code> field, checked before matching	—
Audit trail	Not present	<code>audit_logs</code>	—	—
Operational visibility	Not present	Not present	<code>processing_metrics</code> , rate limits, backup verification	—

Area	v1	v2	v3
Queue technology	Left open	Decided: Postgres queue	—
Recognition implementation	Coupled to dlib	Provider interface	—
Monitoring roster	Every enrolled student in the class	—	—
Third-party integration	None	None	None
Session time boundaries	<code>scheduled_start</code> / <code>scheduled_end</code> used directly for all scoring	—	—
Genuine mid-class exits	Not modeled — a student stepping out for a real reason (emergency, errand) would eventually trigger the same <code>left_early</code> / <code>uncertain</code> flags as a proxy attempt	—	—

13. Closing Note

Across eight review rounds, the architecture has moved from a naive "recognize every enrolled student against the timetable" design to a layered one: external check-in confirmation narrows the roster, quorum detection establishes when the class was genuinely in session, periodic evidence collection builds a presence picture within that real window, a teacher's one-tap permitted-exit marking pre-empts false proxy flags for genuine exits, quality-aware gap checking distinguishes real absence from occlusion for everything else, and ambiguous cases route to a human instead of a guess. Everything in Section 11 stays deferred, including treating external check-in photos as an enrollment source.

Two open real-world tasks before Phase 0 can fully start:

Confirm with the college whether the external check-in device offers an API or only scheduled exports, and get a sample export to see what fields are really available (drives Phase 1.5's exact implementation). Decide the default `capture_buffer_minutes` (15 is a reasonable starting point) with input from how much timetable slippage is actually typical at your college — this can also just be tuned after the first pilot. No further architecture changes are expected before implementation begins. If a future review — human or AI — proposes another structural change, treat that as a signal to start building and revisit after the pilot generates real data, not as a reason to keep revising the document.